

79. Une fonction : donner un nom à un geste

Quand un bloc de code fait UNE chose, on lui donne un nom, et `main()` se lit comme une histoire.

1. Ce qu'il faut comprendre

Une **fonction** est un bloc de code qui porte un nom. `void cadre() { ... }` le **définit** : rien ne s'exécute encore. `cadre();` l'**appelle** : la console saute dans la fonction, exécute son bloc, puis **revient** juste après l'appel.

`void` veut dire « ne rend rien » : la fonction **fait** quelque chose, elle ne **calcule** pas de réponse (ce sera la leçon d'après).

Regarde `main()` : trois lignes qui disent **ce que** fait le programme, sans dire **comment**. Le « comment » est rangé dans les fonctions. C'est la première raison d'exister des fonctions : **la lisibilité**.

La seconde : **ne pas répéter**. `cadre()` est appelée deux fois, avec un effacement entre les deux ; son code n'est écrit qu'une fois. Corriger un bogue dedans le corrige partout.

`appels` est une variable **globale** — déclarée hors de toute fonction : `main` et `cadre` la voient toutes les deux. Chaque appel l'augmente : on **voit** combien de fois on est passé dans la fonction.

À toi : écris une fonction `titre()` qui écrit le nom de ton jeu au milieu, et appelle-la après `cadre()`.

Les fonctions de cette leçon — et le tuto de chacune

- `#include <texte>` → [0.0.2. La fonction texte\(\) — écrire un mot](#)
- `#include <ecran>` → [27.1. La fonction ecran\(\) — éteindre et rallumer l'écran](#)
- `#include <nombre>` → [0.8.3. La fonction nombre\(\) — écrire un nombre](#)

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **777 octets** de cartouche.

Ce que le compilateur y a gravé, nommé — chaque morceau, sa taille, qui l'a demandé — et ce qu'il a laissé, parce que rien ne l'appelle :

Gravé dans la cartouche :

démarrage	52 o	← la console, à l'allumage — la mise en route de la console
cadre()	112 o	← main() — ta fonction
toutEffacer()	36 o	← main() — ta fonction
main()	58 o	← démarrage — ta fonction
VBlank	8 o	← la console, à chaque image — compte les images, 60 fois par seconde
AttendreImage	28 o	← image() — attend l'image suivante
AttendreVBlank	16 o	← ecran() — attend que l'écran soit libre
AttendreAcces	12 o	← EcrireTexte, EcrireNombre — attend de pouvoir écrire une case
AdresseCase	10 o	← texte() — trouve l'adresse d'une case de l'écran
EcrireTexte	10 o	← texte() — écrit un texte, lettre par lettre
EcrireNombre	33 o	← nombre() — écrit un nombre en chiffres
CopierTuiles	85 o	← démarrage — copie les lettres et les dessins en mémoire vidéo
EffacerCarte	14 o	← démarrage — vide les deux cartes de l'écran
Tuiles	272 o	← CopierTuiles — 17 lettres sur 43

```

"#"          1 o ← texte() – un texte gravé
"#"          1 o ← texte() – un texte gravé
"#"          1 o ← texte() – un texte gravé
"#"          1 o ← texte() – un texte gravé
"           ..." 20 o ← texte() – un texte gravé
"APPELS:"    7 o ← texte() – un texte gravé

```

Lettres gravées : AELPS0123456789:#

Pas gravé (jamais appelé) : AvancerAirs, AttendreFinVBlank, EffacerCases, Diviser, Reste, Multiplier, DecalerGauche, DecalerDroite, RangerLutins, InstallerTransfert, Hasard, EffacerFond, EffacerPanneau, LireManette, DonneesTransfert, Notes

Fonctions du langage appelées :

```

texte()      × 6    97 o sur place, dans cadre(), toutEffacer(), main() + AdresseCase (10 o), EcrireTexte (10 o)
ecran()      × 2    21 o sur place, dans main() + AttendreVBlank (16 o)
nombre()     × 1    10 o sur place, dans main() + EcrireNombre (33 o)
image()      × 1    3 o sur place, dans main() + AttendreImage (28 o)

```

Tes fonctions appelées :

```

cadre()      × 2    6 o sur place, dans main()
toutEffacer() × 1    3 o sur place, dans main()

```

1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

```

// CE PROGRAMME : dessine un cadre, efface l'écran, redessine le cadre,
// et affiche combien de fois on a dessiné le cadre.
//
// Ce qui est nouveau : la FONCTION. On donne un nom à un groupe d'instructions,
// puis on l'« appelle » par son nom, autant de fois qu'on veut.
// void cadre() ... = on DÉFINIT la fonction cadre.
// void = « rien » : elle ne rend pas de résultat.
// () vides = on ne lui donne rien.
// cadre();        = on l'APPELLE : ses instructions s'exécutent.

#include <texte>    // écrit un texte à l'écran
#include <ecran>    // éteint ou rallume l'écran
#include <nombre>   // écrit un nombre en chiffres

```

2. UNE VARIABLE — APPELS

```

uint8_t appels = 0;           // Combien de fois cadre() a été appelée.
                              // Déclarée hors des fonctions : toutes la voient.

```

3. UNE FONCTION — CADRE()

```

void cadre() {                // DÉFINITION de cadre : dessine un bord de # tout autour.
    for (uint8_t i = 0; i < 20; i++) { // Les 20 colonnes :
        texte(i, 0, "#");           // en haut (ligne 0)
        texte(i, 17, "#");          // et en bas (ligne 17).
    }
    for (uint8_t l = 1; l < 17; l++) { // Les lignes 1 à 16 :
        texte(0, l, "#");           // à gauche (colonne 0)
        texte(19, l, "#");          // et à droite (colonne 19).
    }
    appels++;                   // Un appel de plus.
}                               // Fin de cadre : on retourne là où on l'a appelée.

```

4. UNE FONCTION — TOUTEFFACER()

```

void toutEffacer() {          // DÉFINITION de toutEffacer : vide tout l'écran.
    for (uint8_t l = 0; l < 18; l++) { // Chaque ligne 0 à 17...
        texte(0, l, "          "); // ... est recouverte de 20 espaces.
    }
}

```

5. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

```
int main() {                                // Le jeu commence ici (pas dans cadre !).
    ecran(0);                               // Dessin écran éteint.
    cadre();                                // 1er appel : le cadre est dessiné. appels = 1.
    toutEffacer();                          // Tout est effacé.
    cadre();                                // 2e appel : le cadre revient. appels = 2.

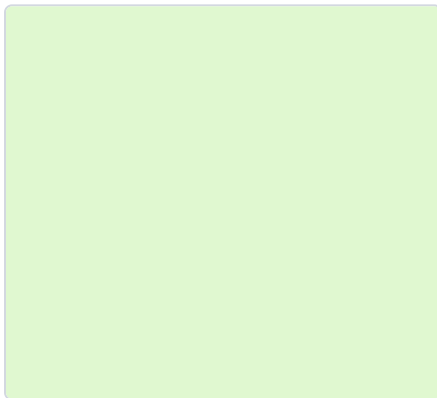
    texte(4, 8, "APPELS:");
    nombre(12, 8, appels);                 // Affiche 002.

    ecran(1);                               // On rallume.

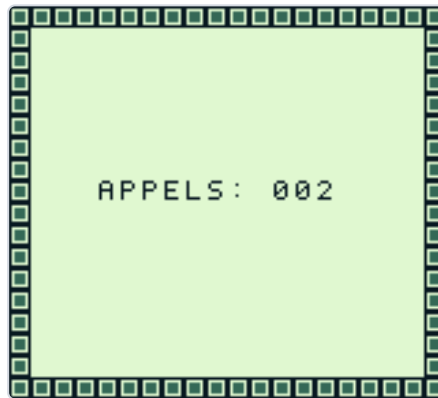
    while (true) {                          // La boucle du jeu.
        image();
    }
}
```

3. Ce qui se passe, étape par étape

La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



la toute première image — la console vient de démarrer — image 1



un tiers de seconde plus tard : le premier dessin est posé — image 20

4. Ce qu'on doit voir

Un cadre de # tout autour de l'écran, et « APPELS: 002 » au milieu.

5. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

- ✓ le bord du haut est dessiné
- ✓ le bord du bas aussi
- ✓ les côtés
- ✓ la fonction a été appelée deux fois

Ensuite : **80. Des arguments : le même geste, ailleurs** — chapitre 12 : Les fonctions : nommer un geste. · [gameboy3](#) — écrire une cartouche Game Boy en C++